

Truck Platooning UPPAAL Model & SVM Implementation



Group (B)

Arya Karimi Jafari

Mohamed Abdo

Mustafa Qaiser

Motivation

- ✓ Driving technology that allows multiple trucks to travel in a column like formation. [2]
- ✓ Improves Fuel Efficiency. [2]
- ✓ Reduces Traffic. [2]
- ✓ Machine Learning role in Automation



Figure : Truck Platooning [2]

SysML Diagrams for Route Change & Configuration

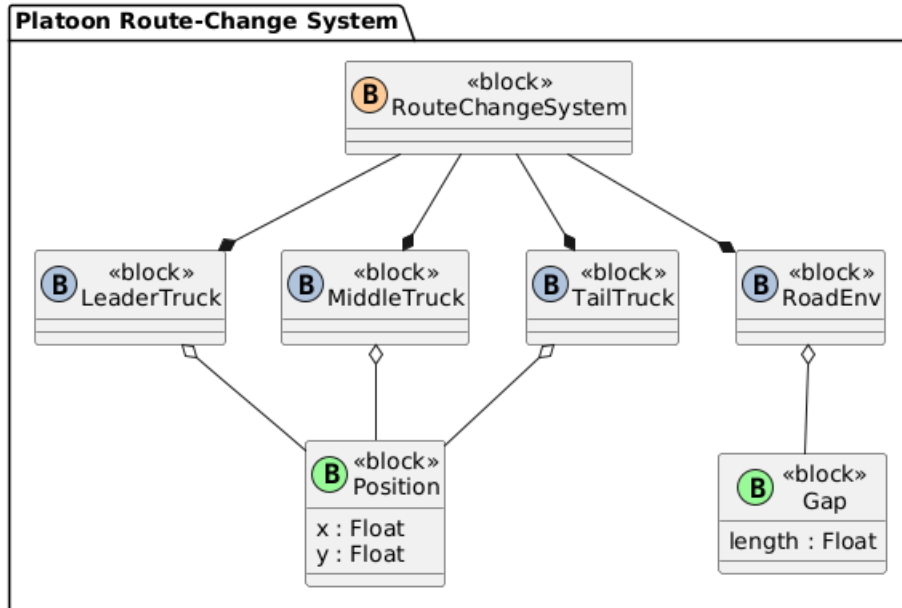


Figure 1.7: Block Definition Diagram for Route Change & Configuration

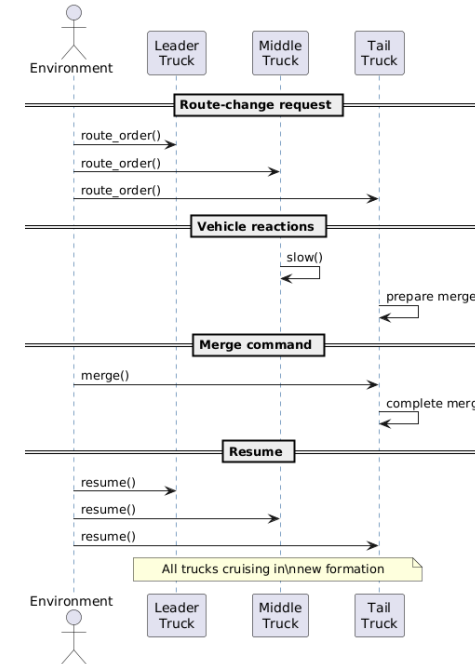


Figure 1.8: Sequence Diagram for Route Change & Configuration

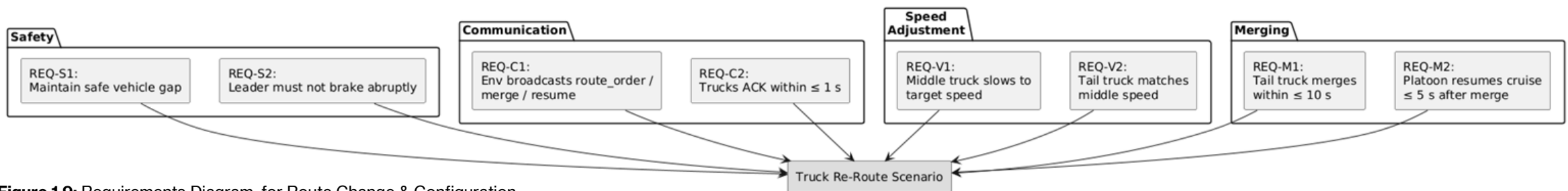


Figure 1.9: Requirements Diagram for Route Change & Configuration

SysML Diagrams for Route Change & Configuration

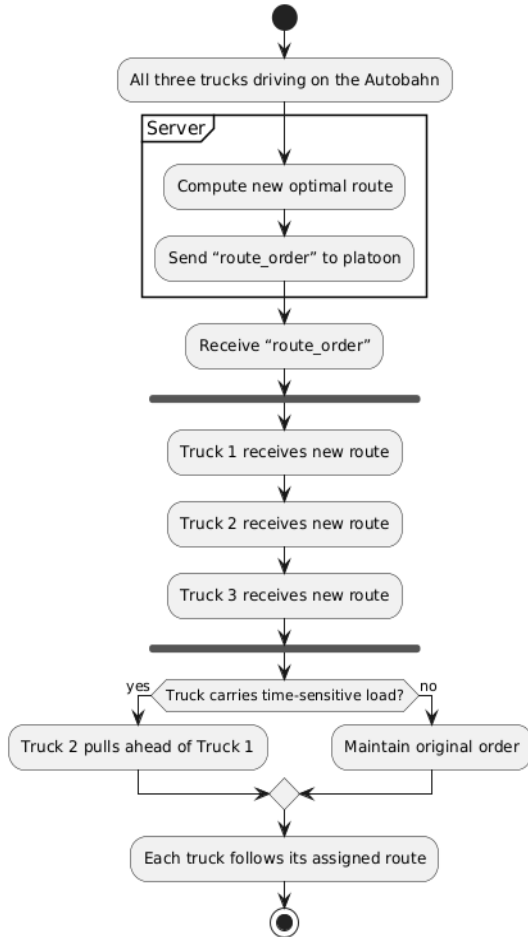


Figure 1.3: Activity Diagram for Route Change & Configuration

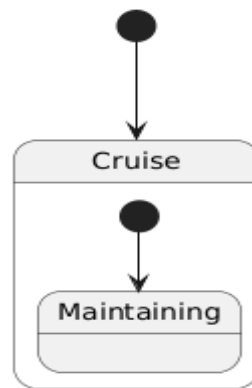


Figure 1.4: State Diagram for Lead Truck

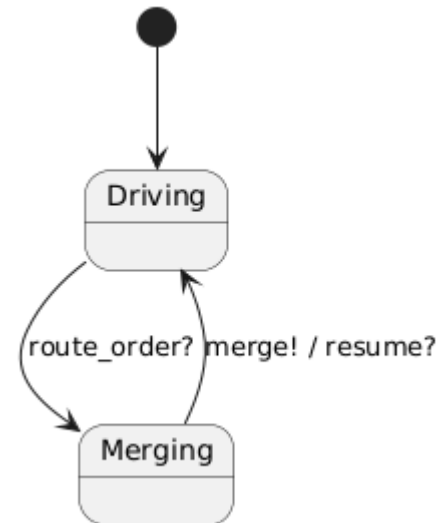


Figure 1.5: State Diagram for Merging Truck

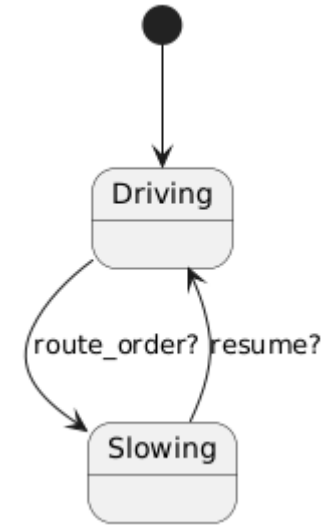


Figure 1.6: State Diagram for Middle Truck

Scenario 1: Route Change & Truck configuration

Declarations:

broadcast chan route_order;
broadcast chan merge;
broadcast chan resume;

System Declarations:

```
env = Env();  
lt = LT();  
mt = MT();  
mg = MG();  
system env, lt, mt, mg;
```

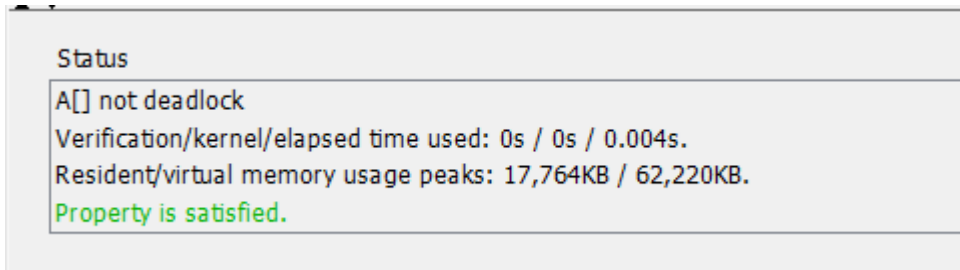


Figure 1.1 Deadlock Checker for Route Change & Configuration Scenario

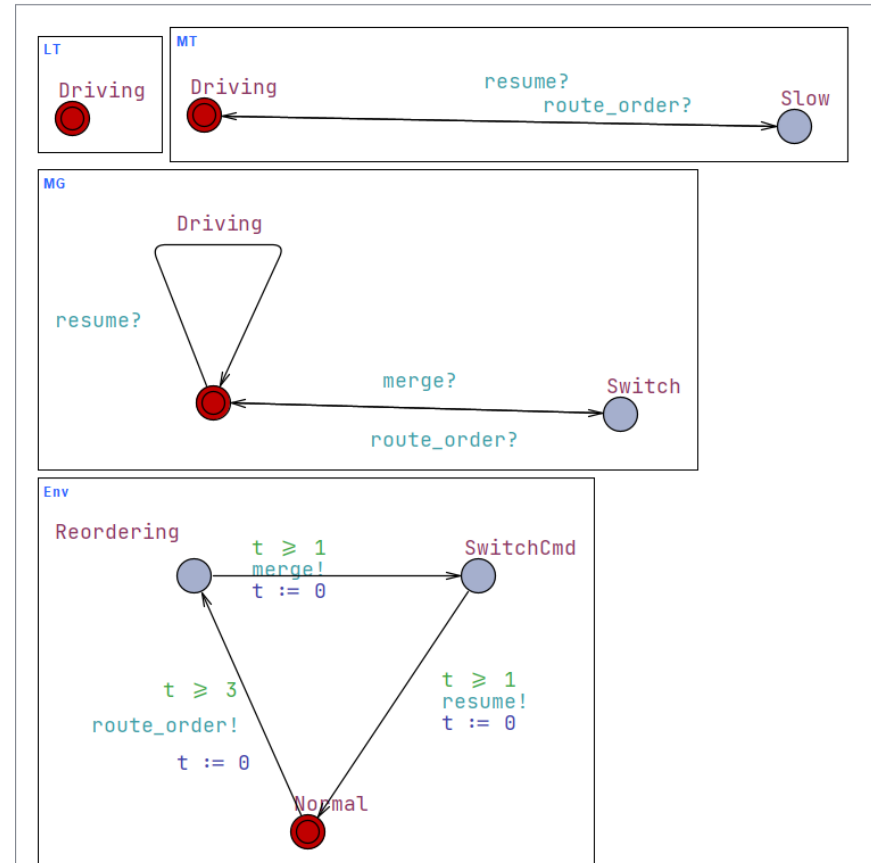


Figure 1.2: Uppaal Diagrams for Route Change & Configuration Scenario

SysML Diagrams for Merging Scenario

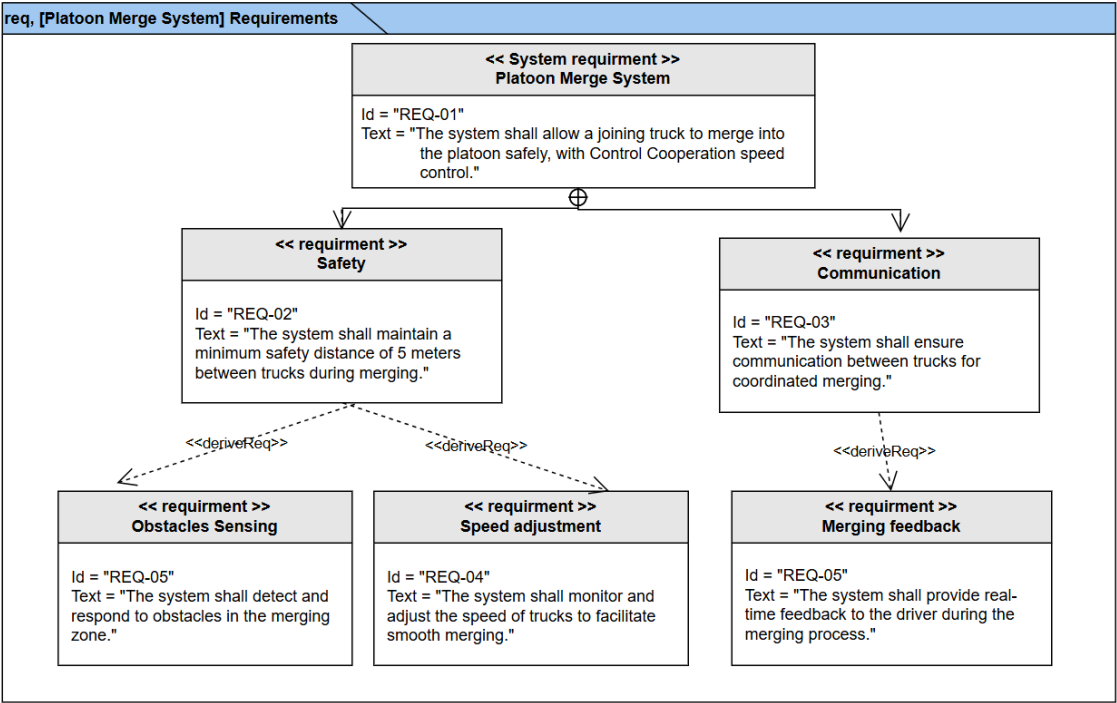


Figure 2.5 : Merging Truck Requirements Diagram

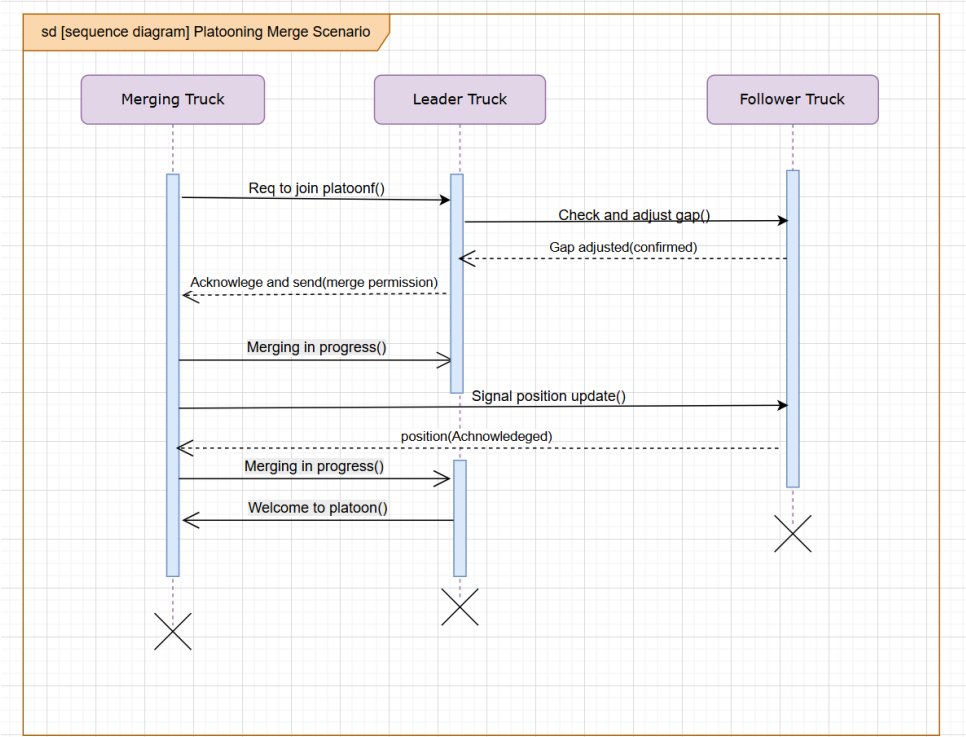


Figure 2.6 : Merging Truck Sequence Diagram

SysML Diagrams for Merging Scenario

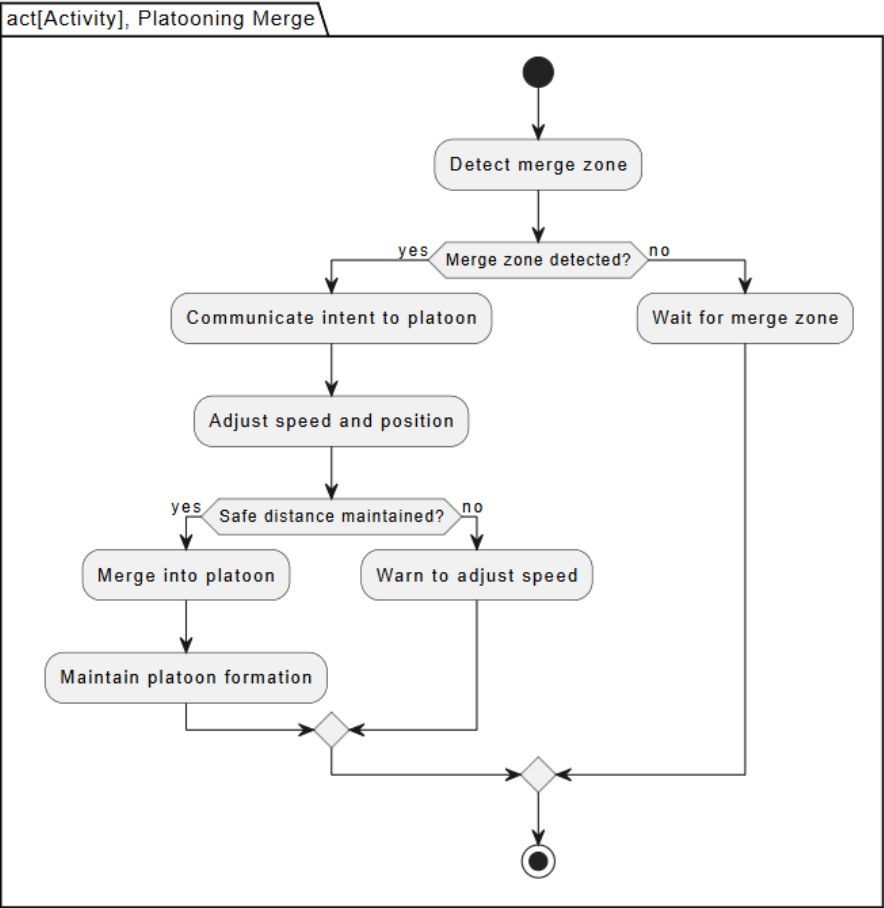


Figure 2.7 : Merging Truck Activity Diagram

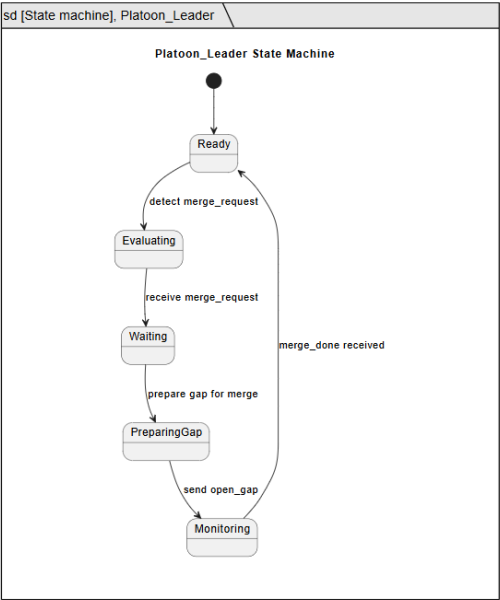


Figure 2.8 : Leader Truck State Chart

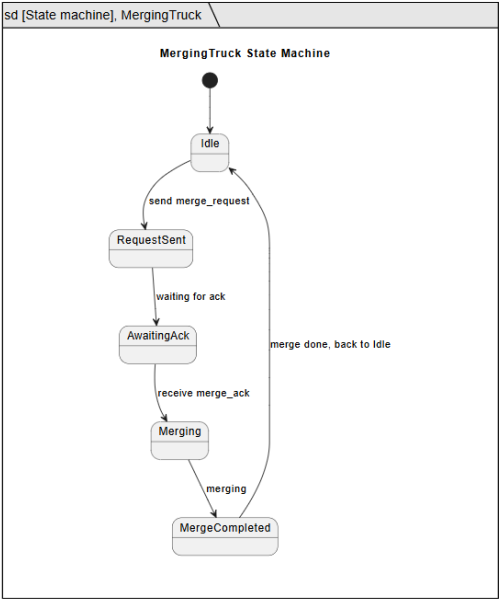


Figure 2.9 : Merging Truck State Chart

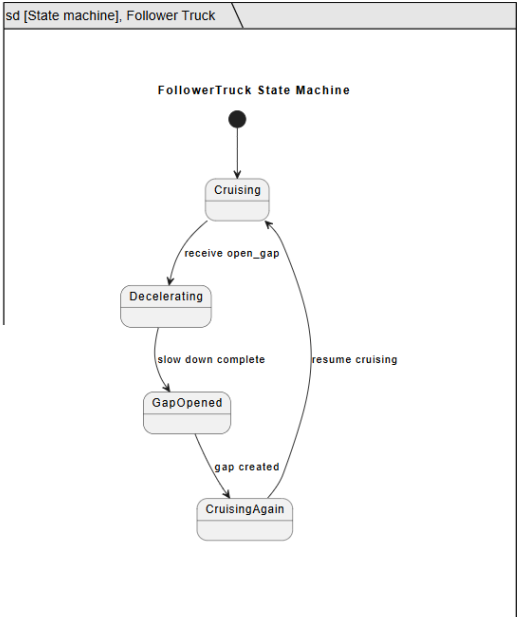


Figure 2.10 : Follower Truck State Chart

Scenario 2 : Merging Truck

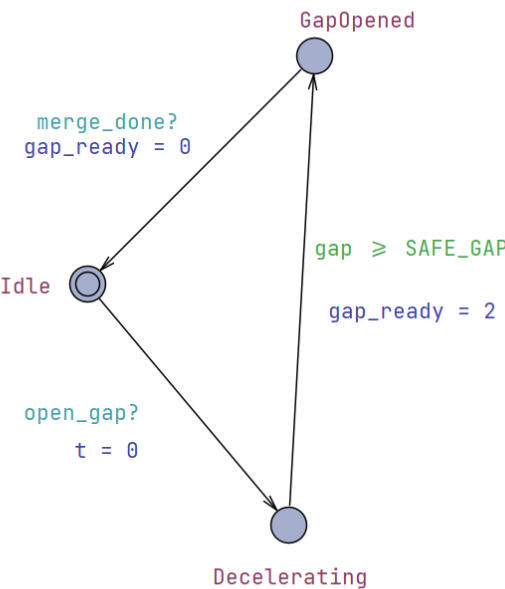


Figure 2.1 : Follower Truck

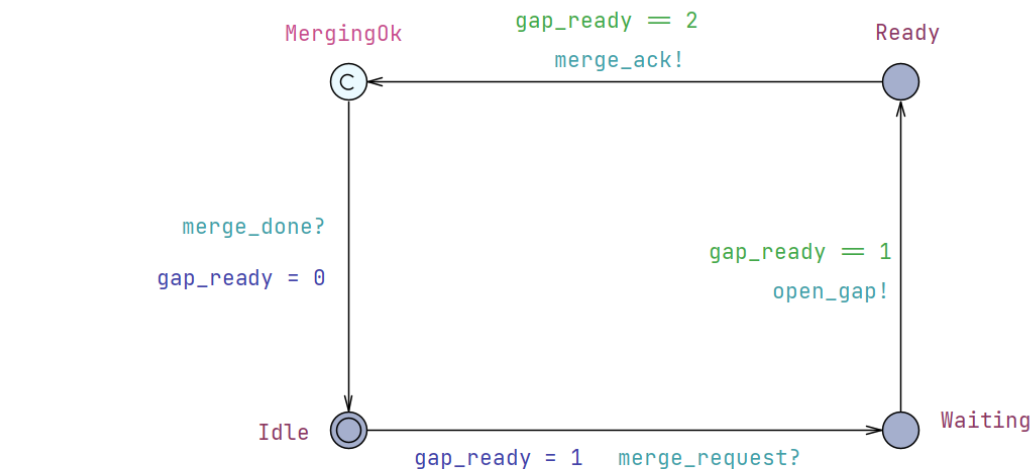


Figure 2.2 : Platoon Leader

```
const int SAFE_GAP = 10;
const int MAX_MERGE_TIME = 10;
const int MAX_RESPONSE_TIME = 5;

int gap_ready = 0;
int gap = 20;
int distance = 0;

broadcast chan merge_request;
broadcast chan merge_ack;
broadcast chan merge_done;
broadcast chan open_gap;
```

Figure 2.4 : Declarations

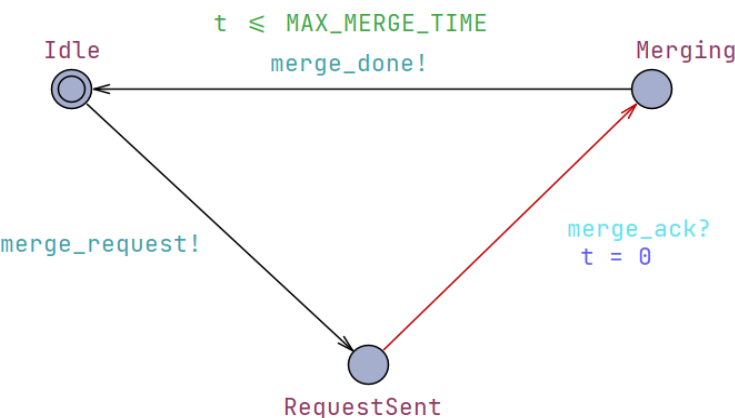


Figure 2.3 : Merging Truck

Status
A[] not deadlock
Verification/kernel/elapsed time used: 0s / 0s / 0.009s.
Resident/virtual memory usage peaks: 18,380KB / 63,664KB.
Property is satisfied.

Figure 2.5 : Deadlock Verifying

Safety Gap & Obstacle Detection SysML Diagrams

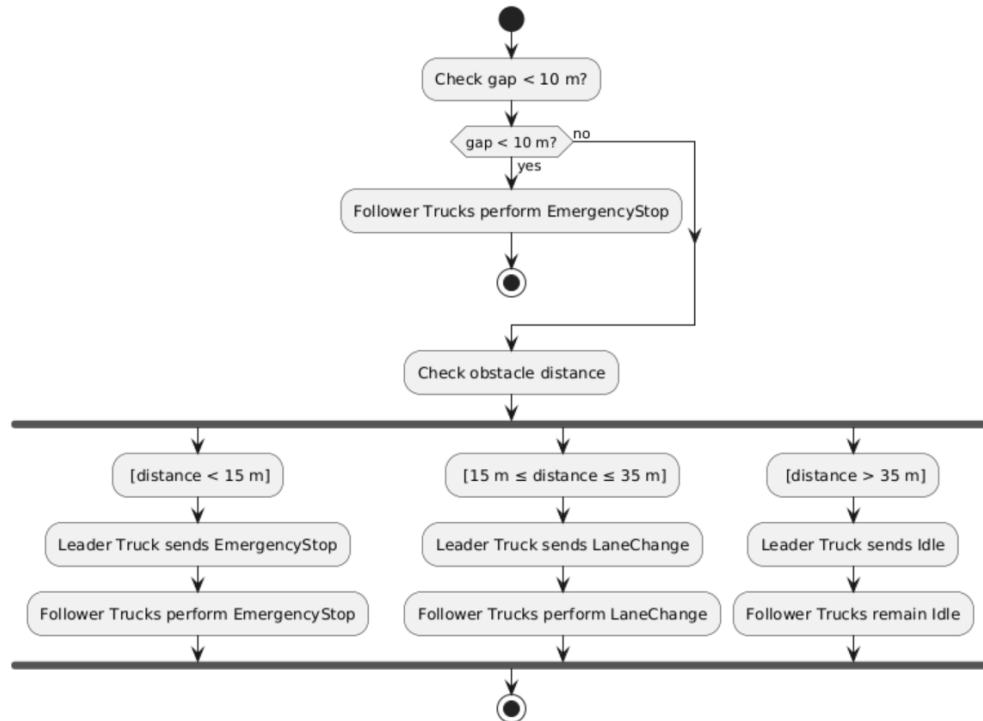


Figure 3.6 : Activity Diagram

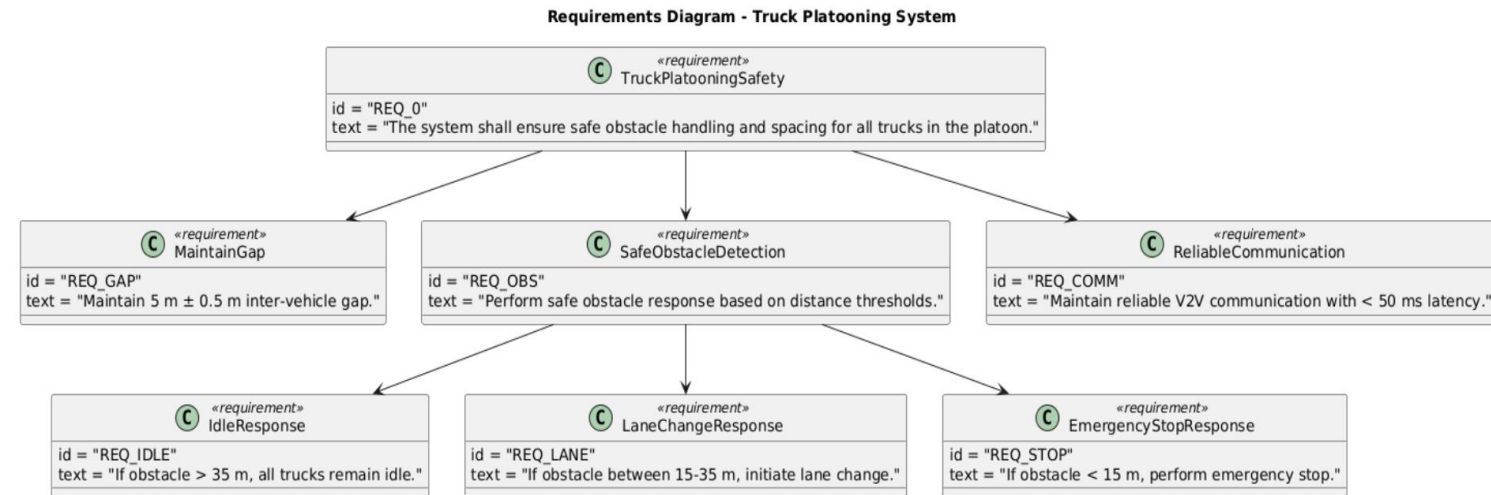


Figure 3.7 : Requirements Diagram

Safety Gap & Obstacle Detection SysML Diagrams

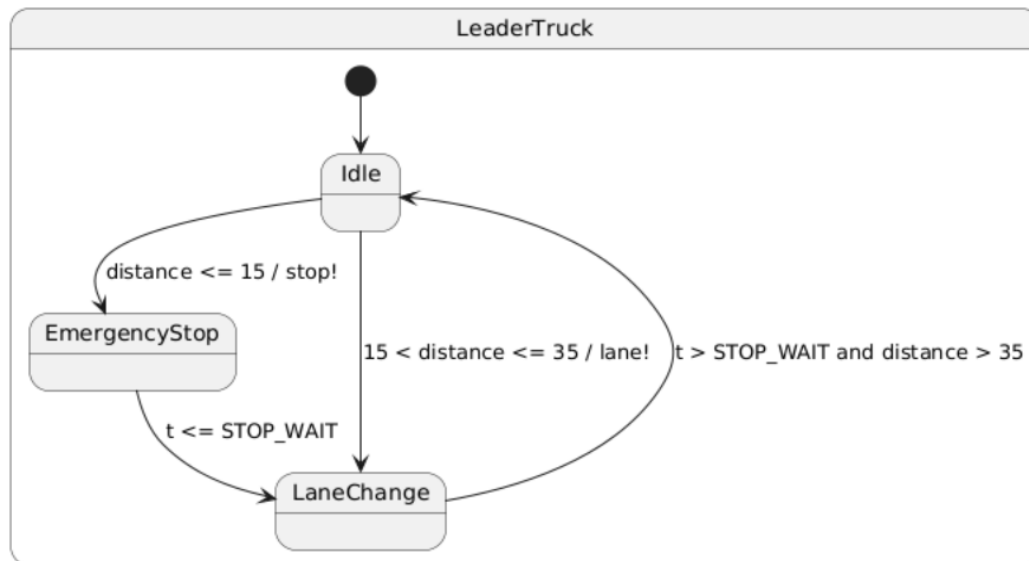


Figure 3.8 : Leader Truck State Charts Diagram

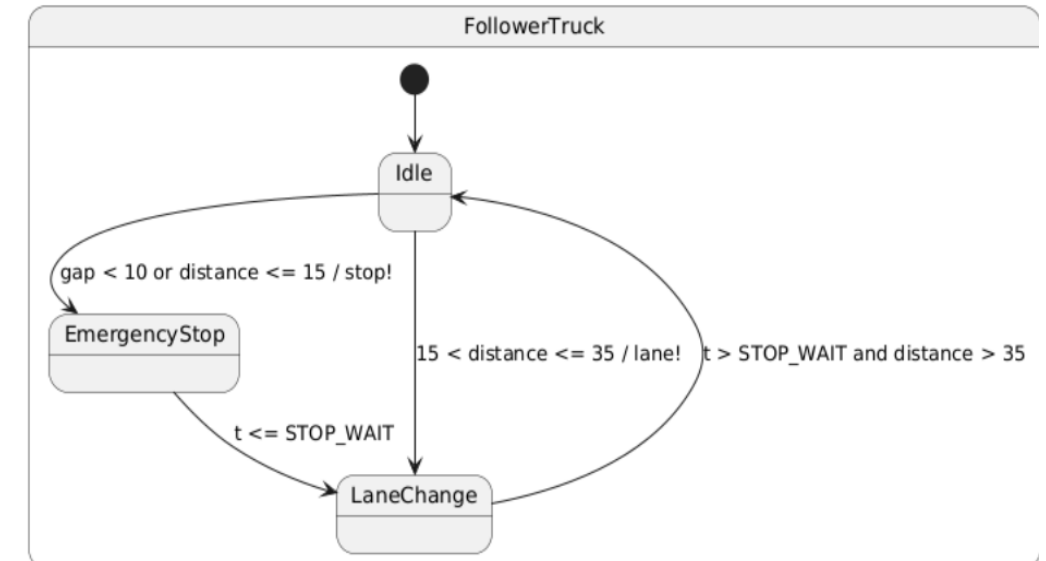
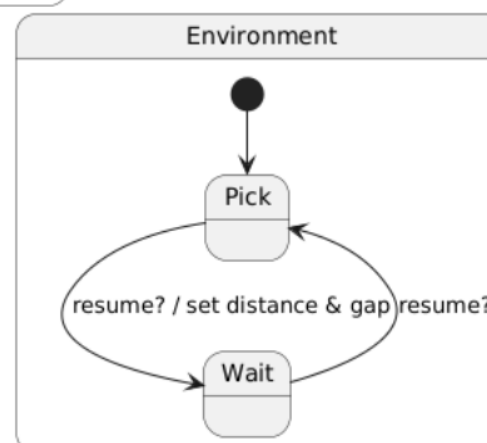


Figure 3.9 : Follower Trucks State Charts Diagram



Scenario 3 : Safety Gap & Obstacle Detection

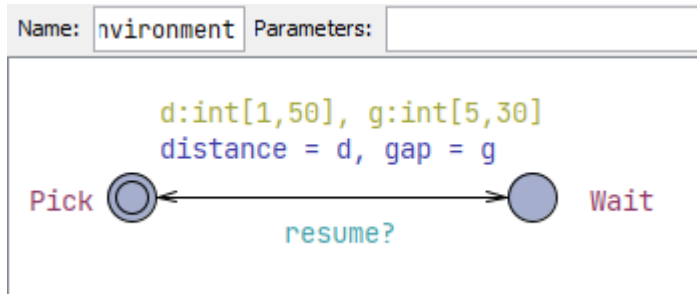


Figure 3.1 : Environment Model

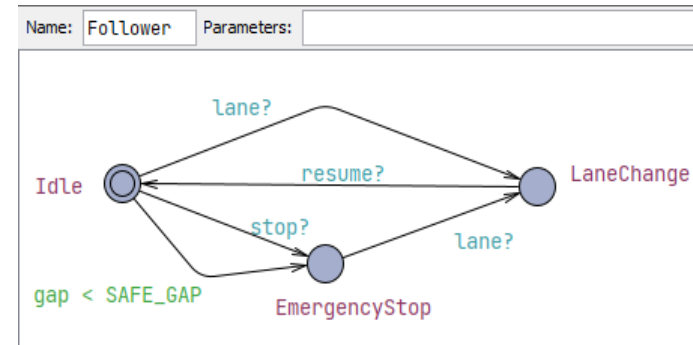


Figure 3.3 : Follower Trucks Model

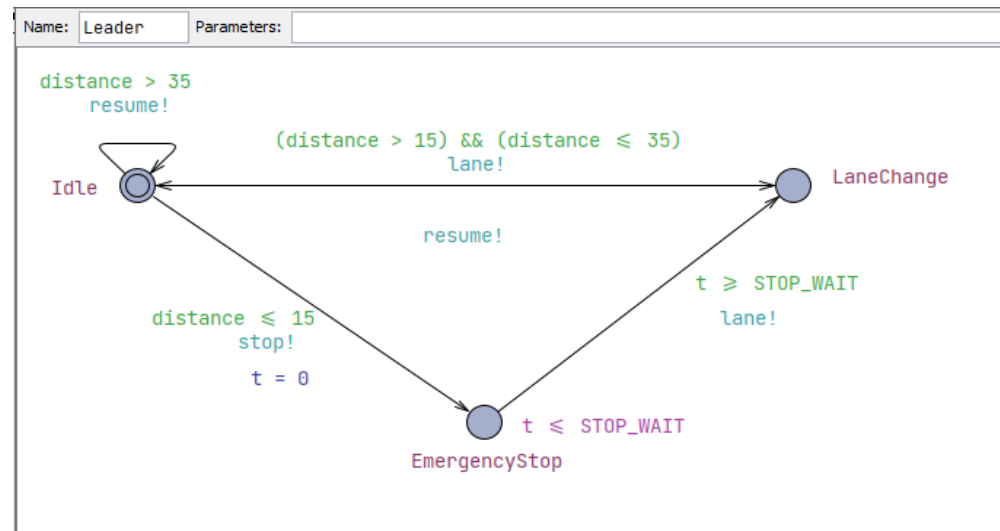


Figure 3.2 : Leader Truck Model

```

const int SAFE_GAP = 10;
const int STOP_WAIT = 5;
int distance = 0;
int gap = 20;
broadcast chan lane;
broadcast chan stop;
broadcast chan resume;

```

Figure 3.4 : Project Declarations

```

env = Environment();
lead = Leader();
f1 = Follower();
f2 = Follower();
system env, lead, f1, f2;

```

Figure 3.5 : System Declarations

The Complete UPPAAL Model

The Complete UPPAAL Model

Environment and Leader Truck

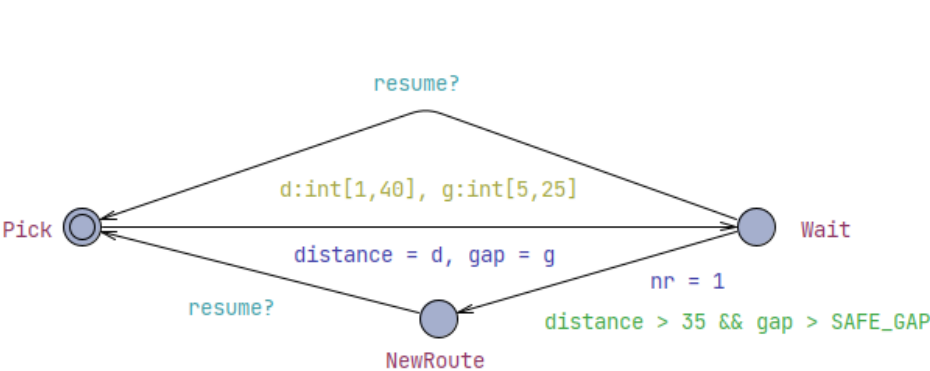


Figure 3 : Environment Model

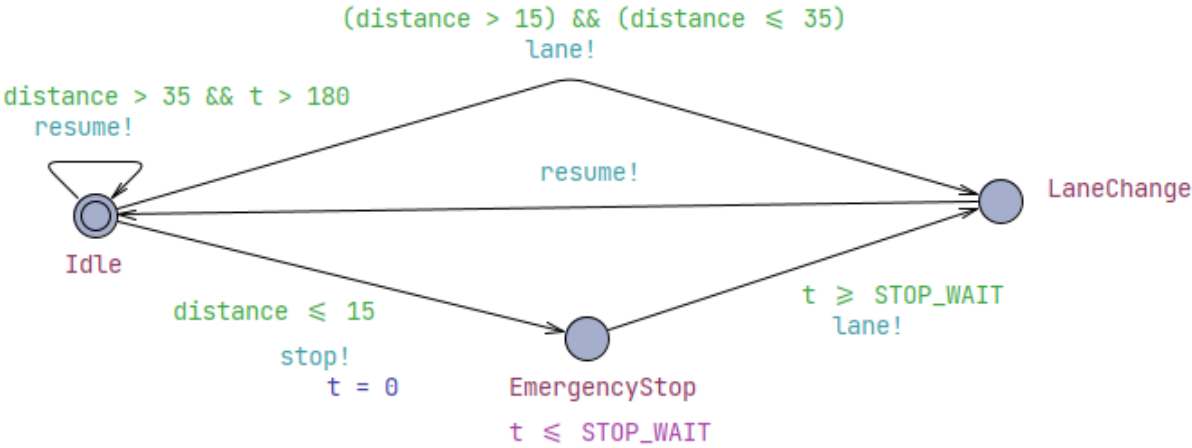


Figure 4: Leader Truck Model

The Complete UPPAAL Model

Follower Trucks

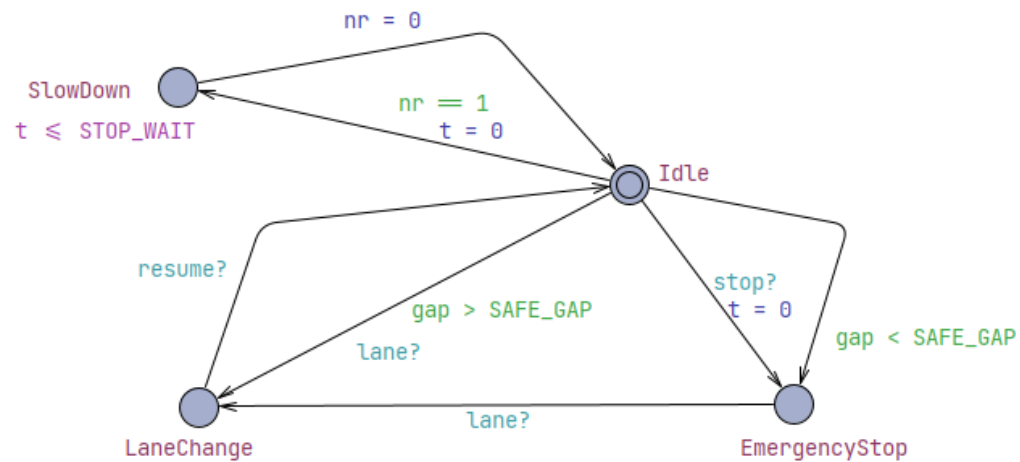


Figure 5: Follower Truck 1 Model

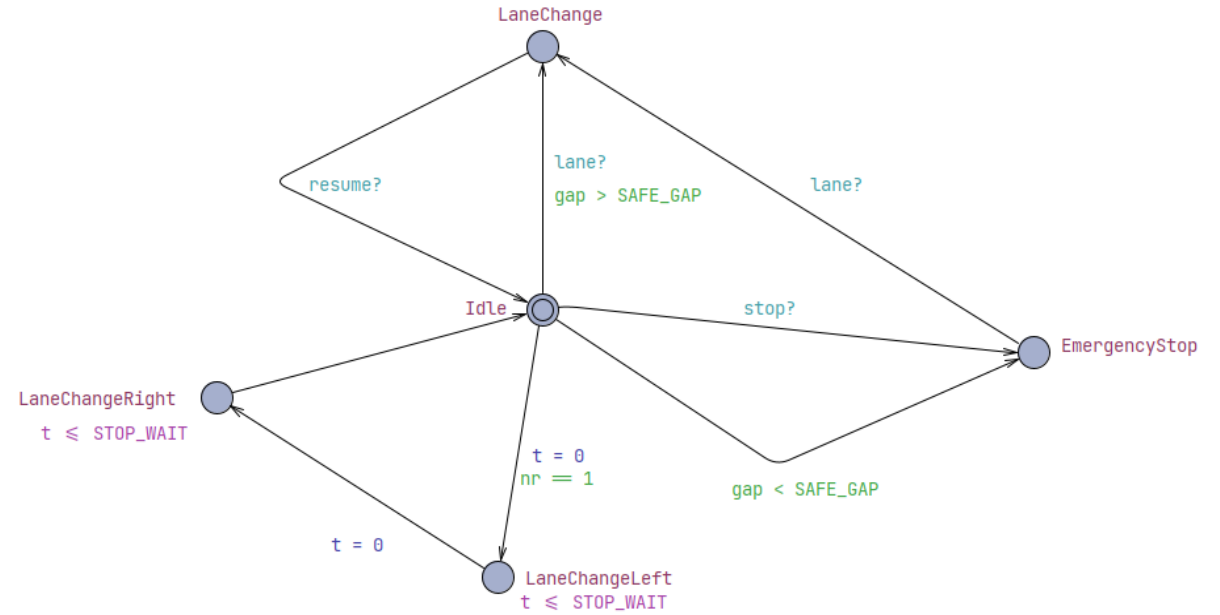


Figure 6: Follower Truck 2 Model

The Complete UPPAAL Model

Merging Truck & Declarations

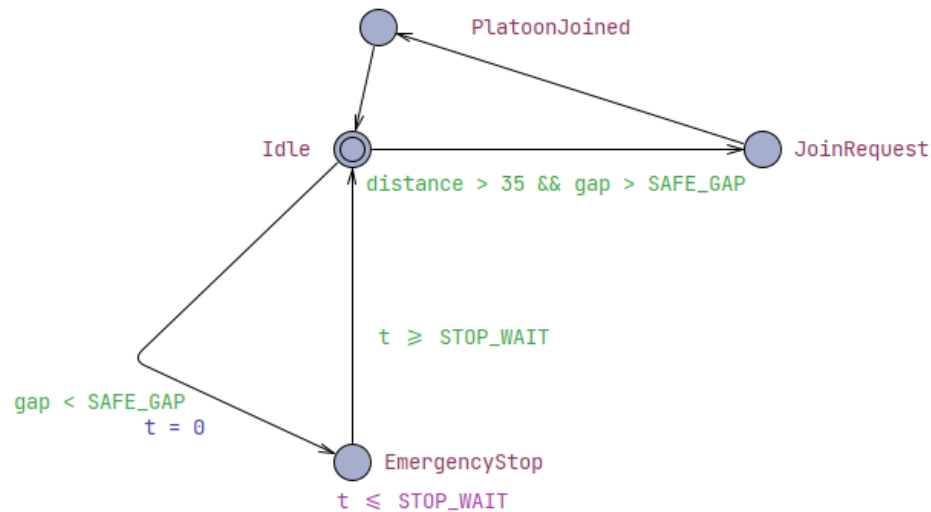


Figure 7: Merging Truck Model

```
const int SAFE_GAP = 10;
const int STOP_WAIT = 5;
int distance = 0;
int gap = 20;
int nr = 0;
broadcast chan lane;
broadcast chan stop;
broadcast chan resume;
```

Figure 8: Complete Declarations

```
env = Environment();
lead = Leader();
f1 = Follower1();
f2 = Follower2();
mt = MergingTruck();
system env, lead, f1, f2, mt;
```

Figure 9: Complete System Declarations

```
A[] not deadlock
Verification/kernel/elapsed time used: 232.125s / 0.516s / 232.906s.
Resident/virtual memory usage peaks: 119,124KB / 262,992KB.
Property is satisfied.
```

Figure 10: Verifying

The Complete UPPAAL Model

Obstacle Detection Sequence Diagram

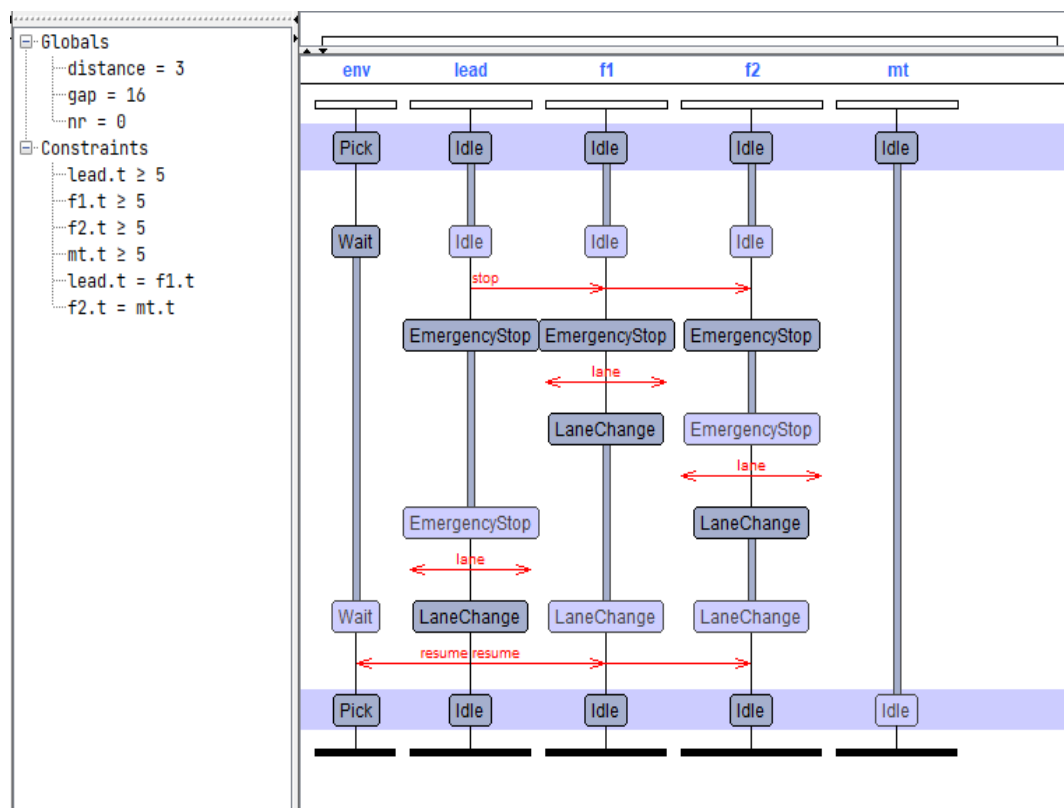


Figure 10 : Obstacle Detection Without Gap Issues

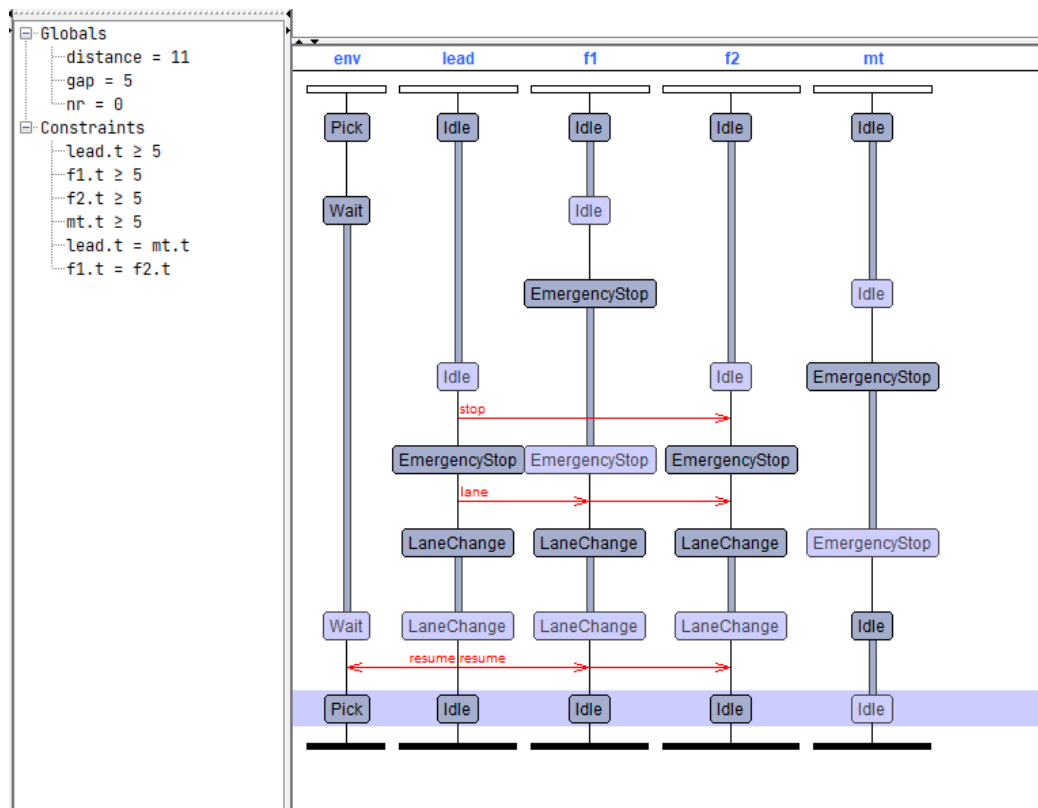


Figure 11: Obstacle Detection With Gap Issues

The Complete UPPAAL Model

Lane Changing Sequence Diagram

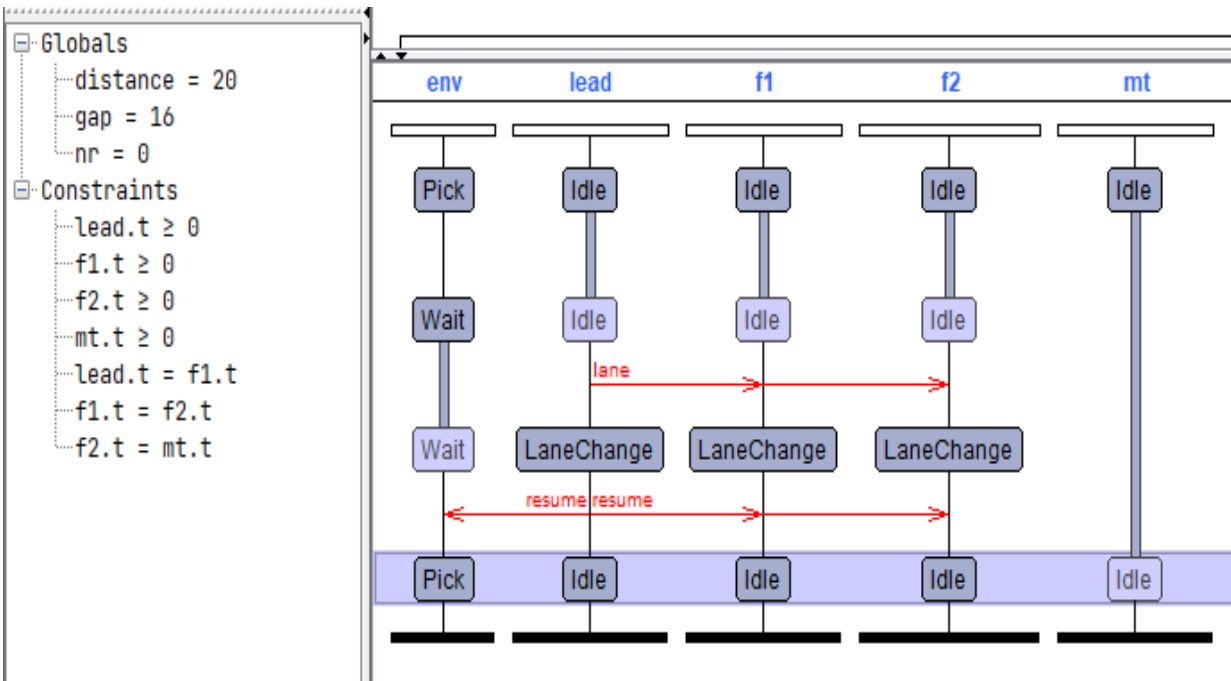


Figure 12: Lane Change Without Gap Issues

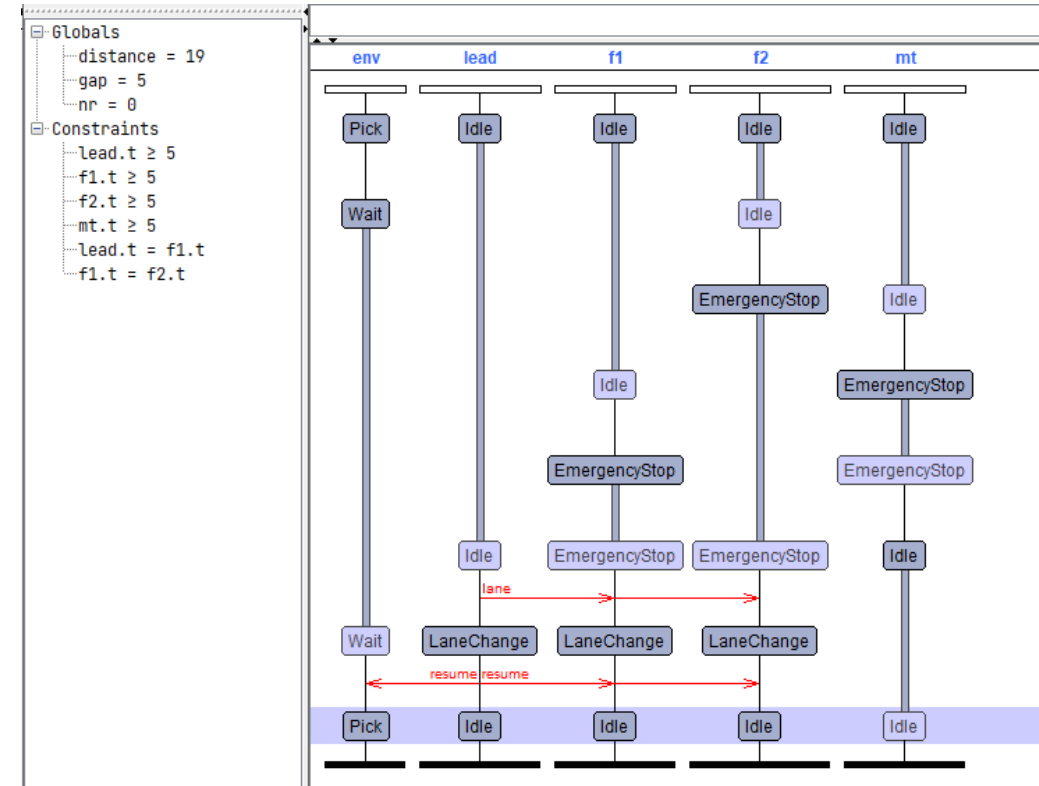


Figure 13: Lane Change With Gap Issues

New Route and Merging Truck



- ✓ The distance to the obstacle & safety gap must be accepted before the new route and merging truck are allowed to be activated.
- ✓ As Follower2 changes lanes, Follower1 slows down, allowing Follower2 to overtake.
- ✓ Merging Truck requests to join the platoon before joining and returning to idle

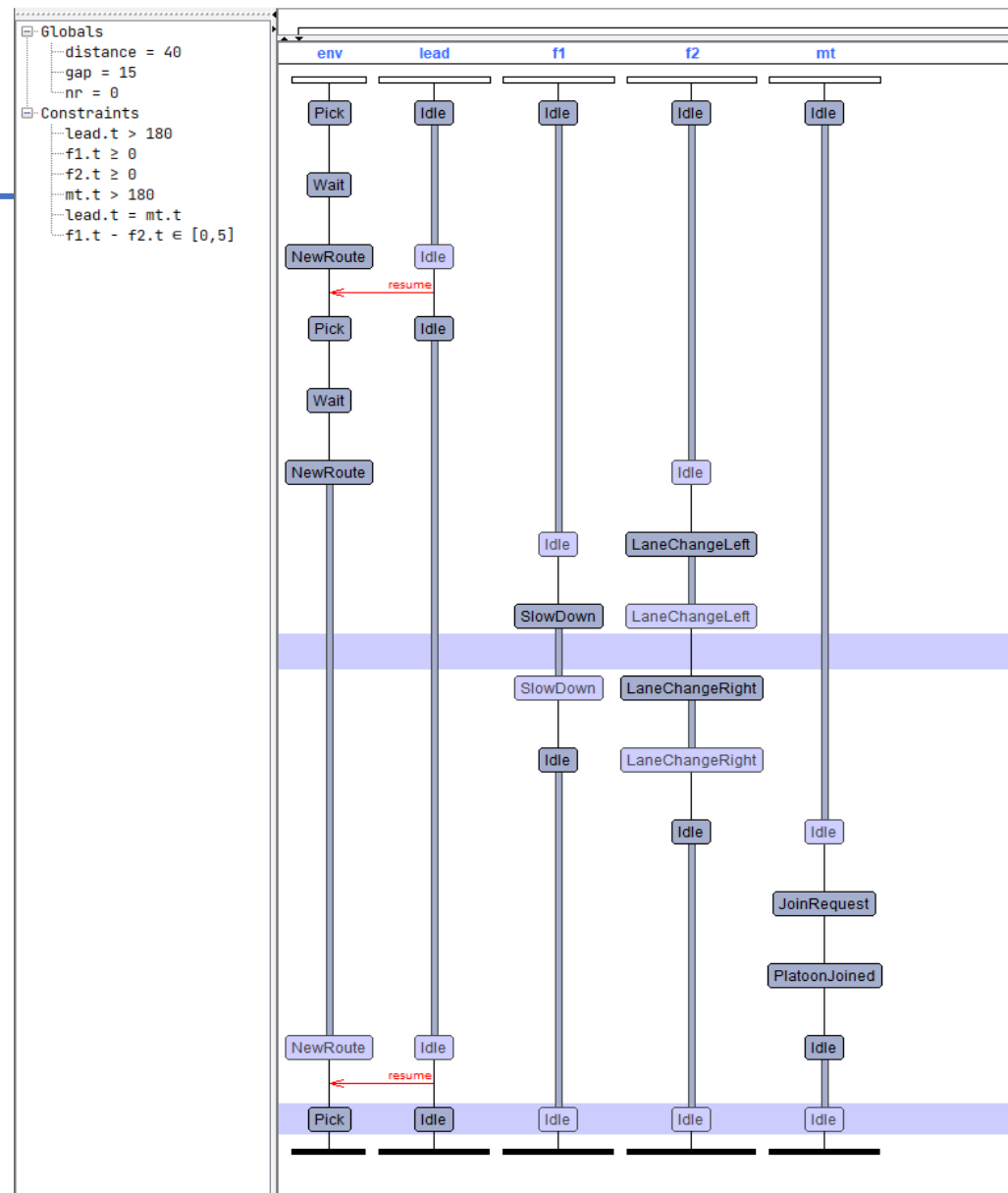


Figure 14: New Route Activation & Merging Truck

SVM Implementation

Carla Dataset Filtering

```
def detect_green_or_blue_lane_center(image):
    h, w, _ = image.shape
    bottom_half = image[int(0.5*h):, :]
    hsv = cv2.cvtColor(bottom_half, cv2.COLOR_BGR2HSV)
    lower_green = np.array([40, 40, 40])
    upper_green = np.array([90, 255, 255])
    lower_blue = np.array([85, 10, 150])
    upper_blue = np.array([115, 150, 255])
    mask_green = cv2.inRange(hsv, lower_green, upper_green)
    mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
    mask = cv2.bitwise_or(mask_green, mask_blue)
    coords = np.column_stack(np.where(mask > 0))
    if coords.size == 0:
        lane_center = None
    else:
        lane_center = int(np.mean(coords[:, 1]))
    img_width = bottom_half.shape[1]
    return lane_center, img_width, mask
```

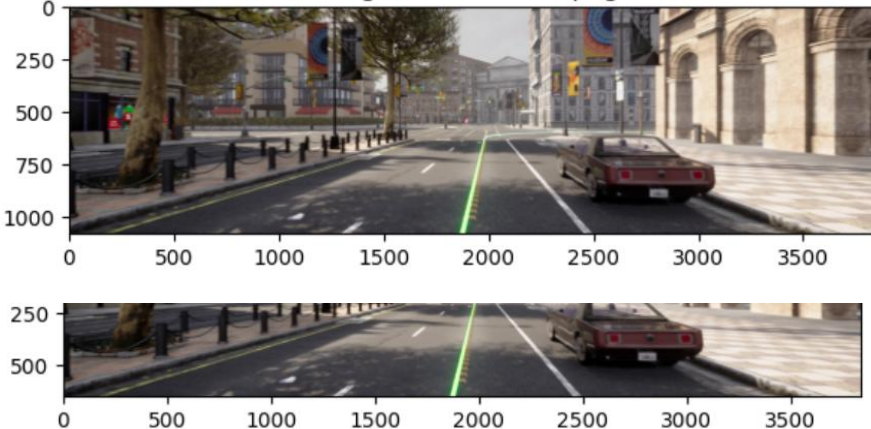
✓ Final Split Summary:

✓ With green line	: 1949
✗ Without green line	: 2259
✓ Centered green line	: 630
↻ Off-center green line	: 1319
✗ Corrupted images	: 0

$$c = \frac{1}{n} \sum_{i=1}^n x_i$$

the average x-position of lane pixels to find the lane center. [4]

Original: 00841055.png



- Green& Blue Lane filtering.
- Focused on **bottom half** of each image for lane visibility
- Converted from **BGR to HSV** color space
- Created a **binary mask** of lane pixels where mask > 0

Classes & SVM Training

```
def classify_lane_position(lane_center, img_width, threshold=0.05):  
    if lane_center is None:  
        return 0.0  
    center_x = img_width // 2  
    delta = lane_center - center_x  
    norm_delta = delta / img_width  
    if norm_delta < -threshold:  
        return -1.0  
    elif norm_delta > threshold:  
        return 1.0  
    else:  
        return 0.0
```

$$\text{distance}_{\text{support vectors}} = \frac{|\beta_0 + \beta^T x|}{\|\beta\|} = \frac{1}{\|\beta\|}.$$

Shows how close points are to the line. SVM makes this distance big [5]

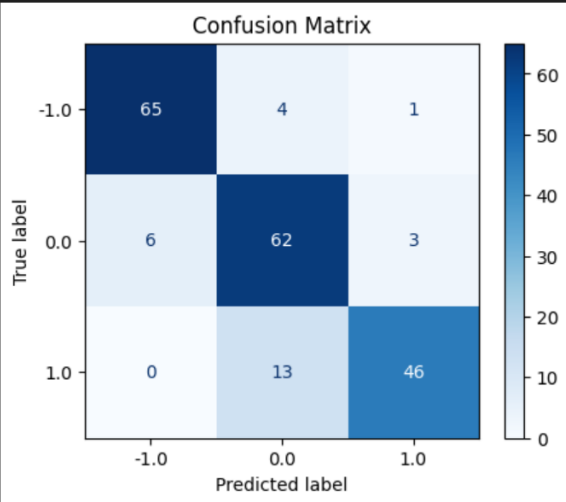
- Lane position is classified into:
Left (-1), Center (0), Right (+1)
- SVM model is trained with: **Linear kernel**
- **Balanced classes** to improve fairness
- SVM find the best hyperplane that separates data into classes

```
df = pd.DataFrame(data)  
df['image'] = df['image'].apply(lambda x: np.array(x, dtype=np.float32))  
train_df, test_df = train_test_split(df, test_size=200, random_state=42, shuffle=True)  
  
X_train = np.vstack(train_df['image'].values)  
y_train = train_df['label'].values  
  
X_test = np.vstack(test_df['image'].values)  
y_test = test_df['label'].values  
  
clf = make_pipeline(StandardScaler(), SVC(kernel='linear', class_weight='balanced', verbose=True, max_iter=2000))
```

- Measures how far a point is from the SVM line
- SV are the closest points & SVM tries to make that distance as large as possible

Evaluation & CARLA Implementation

Test accuracy on 200 held-out images: 86.50%



```
def predict_steering(img):
    if not hasattr(predict_steering, "_model"):
        model_path = MODEL_PATH
        if not os.path.isfile(model_path):
            predict_steering._model = None
        else:
            predict_steering._model = joblib.load(model_path)
    img_np = np.frombuffer(img.raw_data, dtype=np.uint8)
    img_np = img_np.reshape((img.height, img.width, 4))
    img_bgr = img_np[:, :, :3]
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    mask_green = cv2.inRange(img_hsv, lower_green, upper_green)
    mask_blue = cv2.inRange(img_hsv, lower_blue, upper_blue)
    combined_mask = cv2.bitwise_or(mask_green, mask_blue)
    height, width = combined_mask.shape
    bottom_half = combined_mask[int(height * 0.5):, :]
    resized = cv2.resize(bottom_half, IMAGE_SIZE)
    features = resized.flatten().reshape(1, -1)
    model = predict_steering._model
    pred_clipped = 0.0
    if model is not None:
        try:
            pred = float(model.predict(features)[0])
        except Exception as e:
            pred = 0.0
            print(f"[ERR] SVM predict failed: {e}")
        pred_clipped = pred_to_steer_value.get(pred, 0.0)
        print(f"SVM steering prediction: {pred_clipped:.3f}")
    return pred_clipped
```

- Model tested on **200** unseen images
- Achieved **86.5% accuracy**
- Detected green/blue lanes in real-time camera input
- Predicted lane class is used to **steer the vehicle**

Conclusion, Challenges, Future Work

UPPAAL Challenges

- Exporting Timed Automata
- Merging of all scenarios
- Forming coherency

SVMs Challenges

- **Input shape mismatch** caused prediction failures
- Errors from using arrays with **wrong dimensions**
- Needed to carefully **reshape and scale input** before prediction

```
SVM steering prediction: -0.500
[ERR] SVM predict failed: X has 16384 features, but StandardScaler is expecting 1 features a
s input.
SVM steering prediction: -0.500
[ERR] SVM predict failed: X has 16384 features, but StandardScaler is expecting 1 features a
s input.
SVM steering prediction: -0.500
[ERR] SVM predict failed: X has 16384 features, but StandardScaler is expecting 1 features a
s input.
SVM steering prediction: -0.500
```

Error(1): Prediction failed because input shape was too large (16384 instead of 1)

```
righty = int(((vis_img.shape[1] - x) * vy / vx) + y)
SVM predict failed: setting an array element with a sequence. The requested ar
y has an inhomogeneous shape after 1 dimensions. The detected shape was (4,) +
homogeneous part.
/home/iql/MLLine/groupB2.py:58: DeprecationWarning: Conversion of an array with
dim > 0 to a scalar is deprecated, and will error in future. Ensure you extract
single element from your array before performing this operation. (Deprecated Nu
Py 1.25.)
lefty = int((-x * vy / vx) + y)
/home/iql/MLLine/groupB2.py:59: DeprecationWarning: Conversion of an array with
dim > 0 to a scalar is deprecated, and will error in future. Ensure you extract
single element from your array before performing this operation. (Deprecated Nu
Py 1.25.)
```

Error(2): Array shape was incorrect - needed flattening before use

References

[1]: [Stock-Illustration „futuristic trucks fleet on highway with full self driving system activated for transportation autonomy concepts as wide banner with copy space area - Generative AI“ | Adobe Stock](#)

[2]: Rapp AG, "Truck Platooning," Rapp AG. [Online]. Available: <https://www.rapp.ch/de/stories/truck-platooning>. [Accessed: 21-Jun-2025].

[3]: [OpenCV: Contour Features](#)

[4]: [Find Center of a Blob \(Centroid\) Using OpenCV \(C++/Python\) | LearnOpenCV](#)

[5]: [OpenCV: Introduction to Support Vector Machines](#)

Merging Citation:

1- <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9607199>

2- <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10891586#:.:text=In%20essence%2C%20a%20merging%20vehicle%20would%20like,must%20be%20completed%20and%20all%20vehicles%20should>

3- <https://www.sciencedirect.com/science/article/pii/S0968090X21004952#:.:text=Some%20recent%20research%20on%20intersection,highway%20on%20Dramp%20merging%20scenario>.

4- <https://www.sciencedirect.com/science/article/abs/pii/S0191261518311366#:.:text=To%20exploit%20complete%20cooperation%20flexibility,satisfying%20all%20the%20prescribed%20constraints>.

5- <https://pmc.ncbi.nlm.nih.gov/articles/PMC9865798/#:.:text=The%20suggested%20inter%20vehicular%20gap,average%20speed%20of%20the%20platoon>.